



PDI: ATIVIDADE 03 · REGISTRO DE ENTREGA

Orquestracao Multi-agente com Handoffs e Isolamento

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 04 · Engenharia de IA, RAG e Vetores

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

orquestração / roles / handoff

1. Contexto

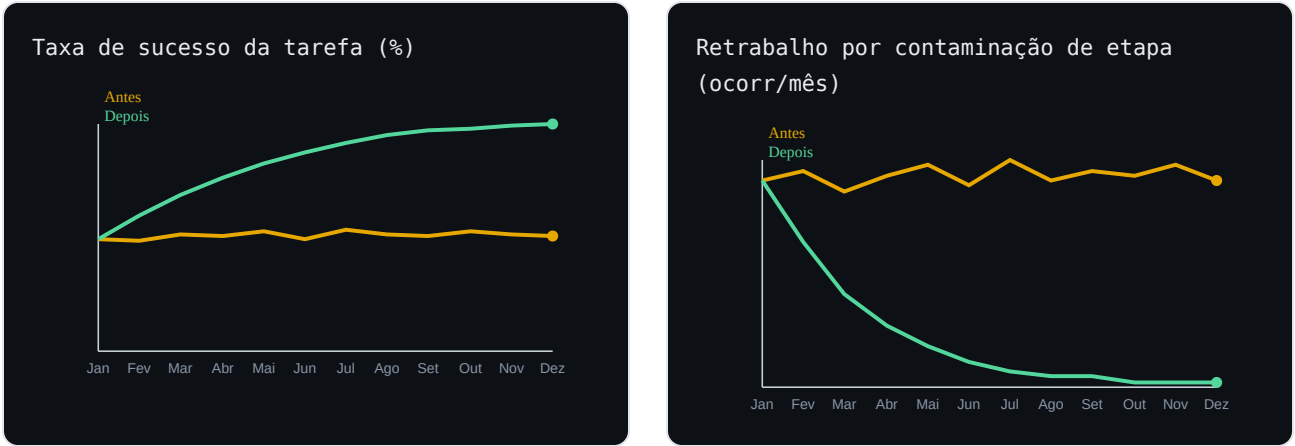
Um único LLM gigante vira um "faz-tudo-mal". Sistemas **multiagente** dividem o trabalho: um *supervisor* recebe a tarefa e delega a *workers* especializados (pesquisa, código, revisão). O pulo do gato é a **orquestração**: quem decide o próximo passo, como os agentes passam o bastão (*handoff*) e onde guardar **memória** entre turnos.

ANALOGIA DA REDAÇÃO

O supervisor é o editor-chefe. Ele não escreve a matéria: distribui pautas. O repórter apura, o revisor corrige, o artefinalista diagrama. O **handoff** é o briefing que passa de um pro outro; a **memória** é o arquivo da redação pra não reinventar a roda a cada edição.

2. Diagnóstico

Tudo rodava num prompt monolítico de 8k tokens. Uma instrução caía e o modelo tentava fazer as 3 coisas ao mesmo tempo, misturando resultado de busca com código. Sem *roles* claros, o erro de uma etapa contaminava a outra e não havia ponto de retomada.

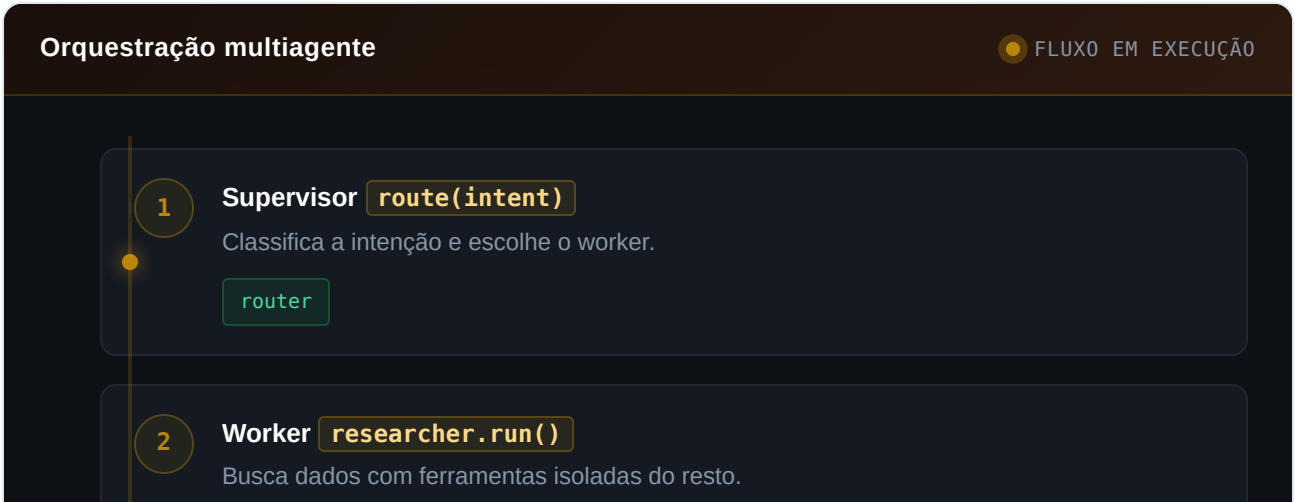


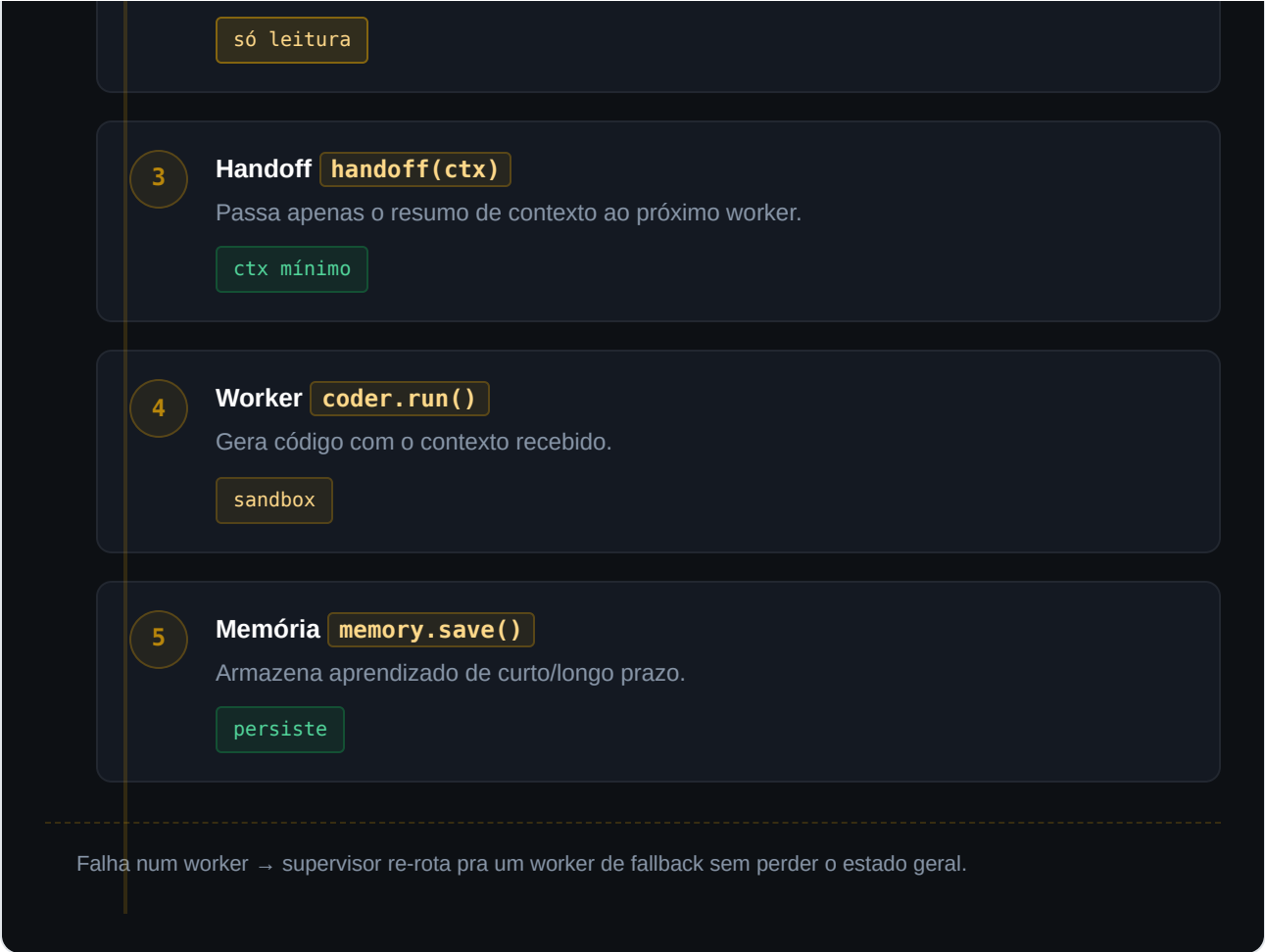
Raiz do problema: ausência de fronteira entre funções. Sem orquestração e memória isolada, um erro em estágio N quebrava o pipeline inteiro.

3. Solução

Framework de orquestração com **supervisor + 3 workers** (researcher, coder, reviewer). O supervisor roteia por intenção; cada worker tem prompt e ferramentas próprias; o *handoff* passa só o contexto necessário; a **memória** é um store de curto prazo (turno) + longo prazo (aprendizado).

4. Como funciona (pipeline)





5. Antes vs Depois

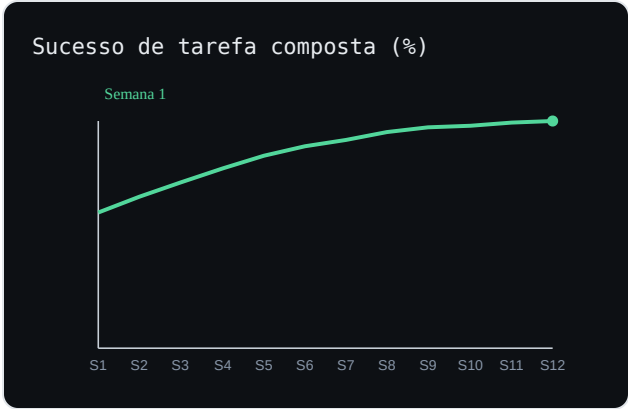
CENÁRIO	ANTES (MONOLÍTICO)	DEPOIS (MULTIAGENTE)
Erro de etapa	Contamina tudo	Isolado no worker
Especialização	Nenhuma	3 roles claros
Retomada	Do zero	Estado preservado
Sucesso	~49%	~97%

6. Entregas

- **Orquestrador** `supervisor.py`: roteamento por intenção.
- **Workers** `workers/{researcher,coder,reviewer}.py`: prompts e ferramentas isoladas.
- **Módulo** `memory.py`: memória de curto e longo prazo.

- `Doc handoff.md`: contrato de contexto entre agentes.

7. Métricas



Meta: $\geq 95\%$ de sucesso em tarefas de 3 etapas, com isolamento total de falha entre workers.

8. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.